

Name : AKASH SAMPATH

Topic : Terms

DNS :

Known as Domain Name System which converts a human friendly name into an Ip address

We remember : google.com and computers need something like IP address such as

142.250.183.14

So google.com uses DNS from IP address to load the website also vice versa

Think of your phone contacts and we save the number : +91 1234567890, So instead of remembering the phone number , we remember by name as MOM

So DNS does similar by accessing google.com by IP address and DNS itself normally doesn't server your website and it answers as "where should I go ? "

IP Address :

An IP address identifies a device or server on a network such as public Ip or private IP and its like a address of a server of device in a network and this the network where to send traffic

So connecting browser with the name of the website is easy than connecting with public IP and thus DNS maps the name to the address

Database :

A database stores and manages structured application data in electricity application having :

Users	Akash House
Bills	500 rs
Payment	400 rs
Customers	Akash Sampath Ashwin Arun
Meter readings	5 rs per current

Therefore is information is stored in MSQL which database storing application or tool

Such as searching or selecting data such as Users : Bills : Payment : Customers : Readings

The MYSQL returns the data based on what we are searching or want to select

Object Storage :

Object storage stores files or objects rather than rows and tables such as : Images . PDF , Videos , Documents , Website files

Therefore AWS object storage service is mainly used in Amazon S3

Databases VS Object Storage :

Imagine your electricity application lets users upload electricity bill in pdf form and instead of using PDF to S3 directly our database stores information about it like :

Users = Akash House
Bills = 500
file url = " s3 – bucket – bills – 500 .pdf"

Finally database stores :

Users
Bills
Payment
Customers
Meter readings

Also S3 Stores :

users – akash_house.pdf
bills – 500.pdf
customer- akash.jpg

Load Balancing :

Suppose we have one sever and where users uses a EC2 , therefore as the users becomes more and more rather than normal limit

The single server becomes overloaded

So the solution is we have to create multiple servers such EC2 – 1 , 2 , 3 and now you need something to distribute requests and that is Load Balancer

So users uses application load balancer by using EC2 – 1 , 2 ,3 to avoid overload and distribute parallel and equally

Suppose 3 users arrive as User A , B , C and load balancer might send :

User A → EC2-1
User B → EC2-2
User C → EC2-3

Another request such as :

User D → EC2 - 1

Therefore exact distribution depends on the load balancer routing and configuration behaviour

You can have multiple servers when there is more traffic and any EC2 fails , the load balancer can stop sending traffic to it if health checks mark it unhealthy

Scaling :

Scaling means increasing or decreasing resources to handle workload , Suppose your application normally has 2 EC2 servers

But during a festival the traffic increase dramatically and you might need 10 more EC2 servers and that is scaling

There are two types of scaling : Horizontal scaling , Vertical scaling

Horizontal scaling is like adding more machines such as : EC2 is used by a user and after one users can use different EC2

Before:

EC2
↑
Users

After:

┌— EC2
└— EC2
Users —+— EC2
└— EC2
└— EC2

Vertical Scaling is like making one machine bigger such as the EC2 as 2 CPU and 4 GB RAM and we can upgrade in EC2 while doing 8 CPU 32 GB RAM

So we have same EC2 but in this we increase the CPU and RAM capacity

So finally **horizontal** is using more computers and vertical is using bigger computer or upgrades

Transaction :

This works within the database and transaction is a group of database operations that should succeed or fail as one unit

So suppose Akash has 10,000 rs and he transfers 2,000 rs to Ashwin

So two operations are happening :

Operation 1 : Remove 2,000rs from Akash : So Akash has 8,000 rs left

Operation 2 : Add 2,000 rs to Ashwin : Finally Ashwin has 12,000 rs totally

If operation 1 succeeds but operation fails : You would get :

Akash = 8,000rs
Rahul = 10,000 rs
Where 2,000 disappeared

So this is how transaction solves by beginning transactions and if something goes wrong it goes to original state or previous state

Consistency :

This means that data should follow the rules and remains valid , Imagine your database says:

Bill Amount = 1,000
Payment Status = PAID

So this is reasonable

But suppose it says :

Bill Amount = 1,000
Payment Status = PAID
Payment Amount = 0

So these are happening based on the application rules that could make data inconsistent

With database consistency where the database or application prevents an invalid state when the rule is : Account balance cannot be negative : Before transaction : Balance : 5,000 : User tries to withdraw : 10,000

Eventual Consistency :

This means different copies of data may be temporarily disagree but they are expected to become consistent after some time

Imagine you have :

Database Server A Database Server B Database Server C

You update data on Database Server A :

Name = Akash

Immediately :

Database Server A - Akash Database Server B – No value / Old value Database Server C - No value / Old value

So after replication :

Database Server A – Akash Database Server B - Akash Database Server C – Akash

So therefore the system becomes consistent eventually